Assignment 4 Report

Author: Luka Karanovic

Date: November 12, 2026

Overview

This report outlines my process of training a multivariable linear regression model for a heavily modified version of a European Space Agency dataset from the textbook *Fundamentals of Machine Learning for Predictive Data Analytics* by John Kelleher, Brian Mac Namee, and Aoife D'Arcy. The goal is to build a multivariable linear regression model for the European Space Agency to predict astronaut oxygen consumption (OXYCON) based on AGE and HEARTRATE. In other words, the model is $OXYCON = w[0] + w[1] * AGE + w[2] * HEARTRATE$.

The report does a preliminary data exploration of the modified dataset, outlines the stopping conditions set for training, and trains the model 9 times, using all combinations of three different sets of weights:

I. w[0] = 0, w[1] = 0 and w[2] = 0;
II. w[0] = -59.5, w[1] = -0.15 and w[2] = 0.60;
III. Randomly picked values between -10 to 10 for w[0], w[1] and w[2].

and three different learning rates α (0.0002, 0.000002, and 0.00000002).

For each training run, it gathers intermediate results at key indices, tracks which stopping condition caused training to terminate, and graphs the overall trend of the error over the number of iterations.

Afterward, a summary of the testing results is presented, along with the best training process and its resulting weights, which is the final model.

Data Exploration

Since I am unfamiliar with the dataset, I decided to do a quick data exploration to get familiar with the dataset.

The first thing to note is that the dataset is extraordinarily small (only 12 entries and 2 descriptive features), meaning it is likely the model will underfit. However, since the dimensions of the table are so small, running thousands of iterations is not expensive at all so we do not have to worry about computational costs.

age

Semantic Meaning: The current age of the astronaut (rounded to the nearest year).

Minimum Value: 34

Maximum Value: 48

Average Value (Mean): 41.33

Spread (Standard Deviation): 4.10

---

heartrate

Semantic Meaning: The average heart rate of an astronaut throughout five minutes of intense physical work (beats per minute).

Minimum Value: 126

Maximum Value: 158

Average Value (Mean): 143.67

Spread (Standard Deviation): 9.89

Oxygen Consumption (Target attribute)

Semantic Meaning: The amount of oxygen consumed by an astronaut throughout five minutes of intense physical work (likely mL/kg/min based on values).

Minimum Value: 27.07

Maximum Value: 54.85

Average Value (Mean): 38.39

Spread (Standard Deviation): 8.39

Training

Training Process

Here, our goal is to find optimal weights w[0], w[1], and w[2], for the model equation $OXYCON = w[0] + w[1] * AGE + w[2] * HEARTRATE$.

We will train the model **9 times** with **different hyperparameters** (starting weights and learning rate). The 9 runs will be as follows:

Run ID	Starting Weights	Learning Rate
1	w[0] = 0, w[1] = 0, w[2] = 0	0.0002
2	w[0] = 0, w[1] = 0, w[2] = 0	0.000002
3	w[0] = 0, w[1] = 0, w[2] = 0	0.00000002
4	w[0] = -59.5, w[1] = -0.15, w[2] = 0.60	0.0002
5	w[0] = -59.5, w[1] = -0.15, w[2] = 0.60	0.000002
6	w[0] = -59.5, w[1] = -0.15, w[2] = 0.60	0.00000002
7	w[0] = -2.51, w[1] = 9.01, w[2] 4.64	0.0002
8	w[0] = -2.51, w[1] = 9.01, w[2] 4.64	0.000002
9	w[0] = -2.51, w[1] = 9.01, w[2] 4.64	0.00000002

- *Runs 7-9 used programmatically-generated random values between -10 and 10 for the initial weights*

The following are the **<u>stop conditions</u>** for training:

1. **Convergence:** Error is not improving with each iteration (found minimum) $\Delta SSE < 10^{-5}$
    a. In floating-point arithmetic, gradient descent rarely hits exactly zero (due to rounding/truncating), so we assign a really small number to terminate training when the improvements become negligible.
2. **Divergence:** Error explodes towards infinity ($SSE \geq 10^{9}$)
    a. Because the input features (AGE and HEARTRATE) are unscaled and have high values, random weight initializations between -10 and 10 that end up in poor positions could either produce high starting errors, or cause the gradients to be massive which will make errors blow up to infinity (if the starting position is on a steep slope).
3. **Max Iterations:** Training reached $50,000$ steps/iterations.

a. Because the dataset only has 12 rows, running 50 000 batch iterations takes a fraction of a second. This high value gives super small learning rates (like 0.00000002) enough time to gradually reduce towards the local minimum, but not so long that a slow-moving run that doesn't meet the convergence stopping condition holds up the program.

Training Results

For each training run, we will gather intermediate results at key indices, track which stopping condition caused the training process to terminate, graph the overall trend of the error over the number of iterations (with matplotlib), display the final weights, and determine whether this model is acceptable or not.

- The key iterations where the SSE is tracked are (if the model did not terminate before that point):
  - 0, 1, 5, 10, 50, 100, 500, 1000, 5000, 10000, 25000, the last training iteration

Run 1 (weights=[0.0, 0.0, 0.0]) (LR=0.0002)

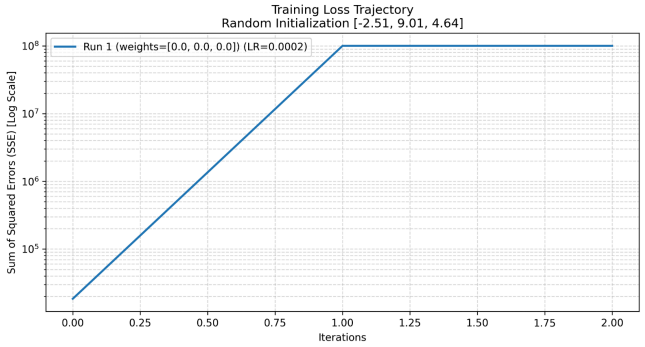
Iteration (0-indexed)	**Sum of Squared Errors (SSE)**
0	18459.4523
1	205174192.3992
2	2334389489038.7817

Stopping Condition: Divergence: $SSE \geq 10^9$ the point oscillated and started rising to infinity.

Final Weights: w[0] = -19.63, w[1] = -811.16, w[2] = -2830.69

Acceptable or not: This model is definitely not acceptable. It's SSE diverged to infinity and it's weights are very low.

Graph:



Run 2 (weights=[0.0, 0.0, 0.0]) (LR=0.000002)

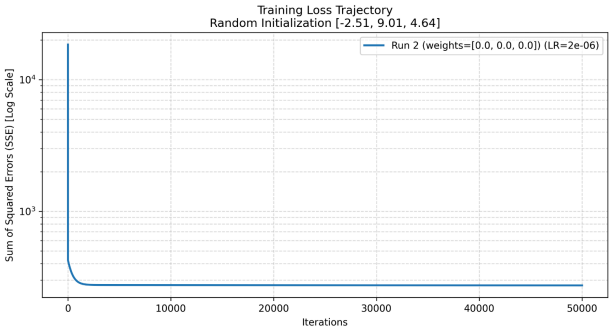
Iteration (0-indexed)	**Sum of Squared Errors (SSE)**
0	18459.4523
1	531.9715
5	424.6012
10	422.8741
50	409.7582
100	394.9925
500	322.5467
1000	290.1643
5000	275.3406
10000	275.1385
25000	274.5378
49999	273.5414

Stopping Condition: Max Iterations Reached (50,000 iterations). The model was approaching the minimum, but improving too slowly.

Final Weights: w[0] = -0.45, w[1] = -0.62, w[2] = 0.45

Acceptable or not: This model is somewhat acceptable. It does not have the lowest error but the error is pretty small and it was converging. Not optimal though.

Graph:



Run 3 (weights=[0.0, 0.0, 0.0]) (LR=0.00000002)

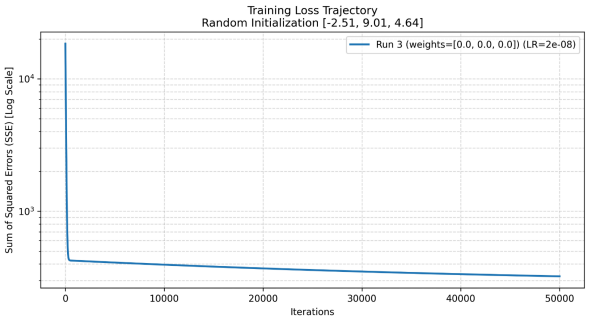
Iteration (0-indexed)	**Sum of Squared Errors (SSE)**
0	18459.4523
1	18073.2290
5	16609.3009
10	14948.9668
50	6534.8608
100	2495.3068
500	424.9610
1000	422.8760
5000	409.7672
10000	395.0085
25000	359.7725
49999	322.5794

Stopping Condition: Max Iterations Reached (50,000 iterations). The model was approaching the minimum, but improving too slowly.

Final Weights: w[0] = 0, w[1] = -0.23, w[2] = 0.34

Acceptable or not: This model is somewhat acceptable. It does not have the lowest error but the error is pretty small and it was converging. Not optimal though.

Graph:



Run 4 (weights=[-59.5, -0.15, 0.6]) (LR=0.0002)

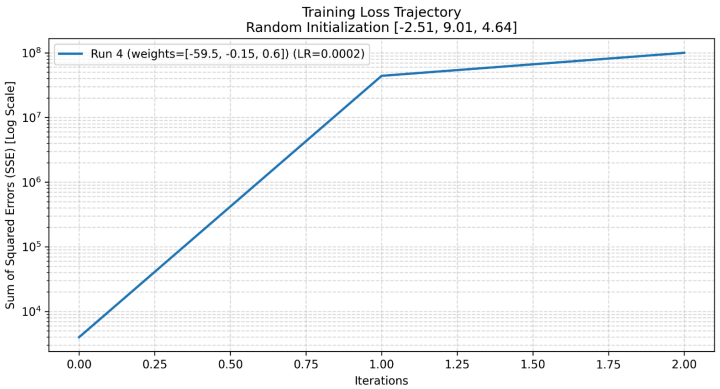
Iteration (0-indexed)	Sum of Squared Errors (SSE)
0	3985.2543
1	44049631.2649
2	501178665765.4503

Stopping Condition: Divergence: $SSE \geq 10^9$ the point oscillated and started rising to infinity.

Final Weights: w[0] = -68.59, w[1] = -375.93, w[2] = -1311.02

Acceptable or not: This model is definitely not acceptable. It's SSE diverged to infinity and it's weights are very low.

Graph (only 2 iterations):



Run 5 (weights=[-59.5, -0.15, 0.6]) (LR=0.000002)

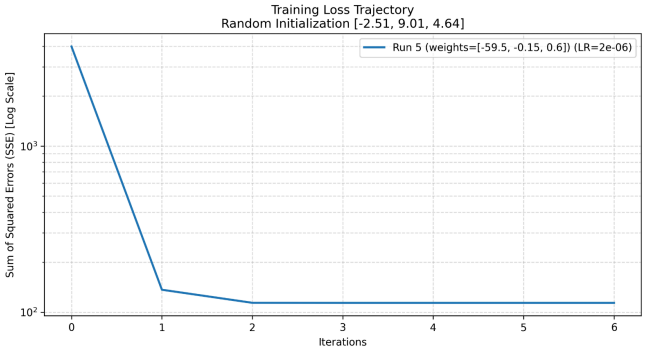
Iteration (0-indexed)	Sum of Squared Errors (SSE)
0	3985.2543
1	136.4129
5	113.6609
6	113.6609

Stopping Condition: Error Converged ($\Delta SSE < 10^{-5}$). The model found a minimum.

Final Weights: w[0] = -59.5, w[1] = –0.12, w[2] = 0.72

Acceptable or not: This model is absolutely acceptable. It converged very early (meaning it found a minimum) and has a small error (the smallest of all runs).

Graph (only 6 iterations):



Run 6 (weights=[-59.5, -0.15, 0.6]) (LR=0.00000002)

Iteration (0-indexed)	Sum of Squared Errors (SSE)
0	3985.2543
1	3902.3354
5	3588.0423

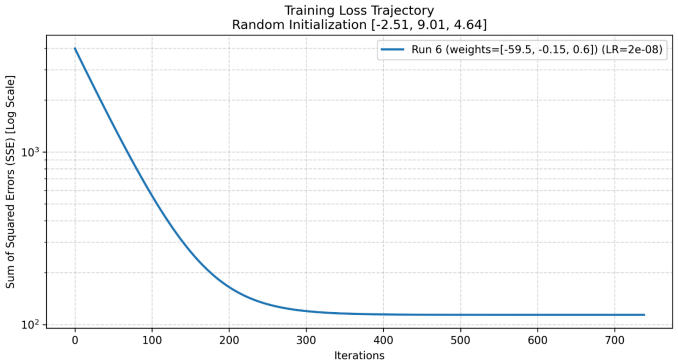
Iteration (0-indexed)	Sum of Squared Errors (SSE)
10	3231.5828
50	1425.1576
100	557.9285
500	113.7379
738	113.6613

Stopping Condition: Error Converged ($\Delta SSE < 10^{-5}$). The model found a minimum.

Final Weights: w[0] = -59.5, w[1] = –0.12, w[2] = 0.72

Acceptable or not: This model is absolutely acceptable. It converged very early (meaning it found a minimum) and has a small error.

Graph:



Run 7 (weights=[-2.51, 9.01, 4.64]) (LR=0.0002)

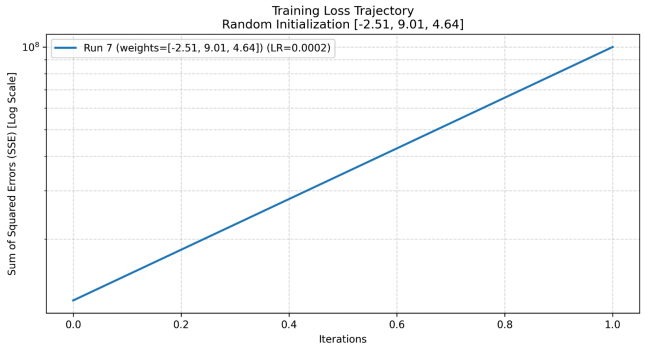
Iteration (0-indexed)	Sum of Squared Errors (SSE)
0	11981078.1953
1	136096592649.3385

Stopping Condition: Divergence: $SSE \geq 10^9$ the point oscillated and started rising to infinity.

Final Weights: w[0] = -7.30, w[1] = -189.57, w[2] = -685.07

Acceptable or not: This model is definitely not acceptable. It's SSE diverged to infinity and it's weights are very low.

Graph (only 1 iteration):



Run 8 (weights=[-2.51, 9.01, 4.64]) (LR=0.000002)

Iteration (0-indexed)	**Sum of Squared Errors (SSE)**
0	11981078.1953
1	89549.6887
5	19078.3716
10	18860.4012
50	17205.1324
100	15341.6742
500	6199.6725
1000	2114.9265
5000	264.0312
10000	263.6786
25000	263.1186

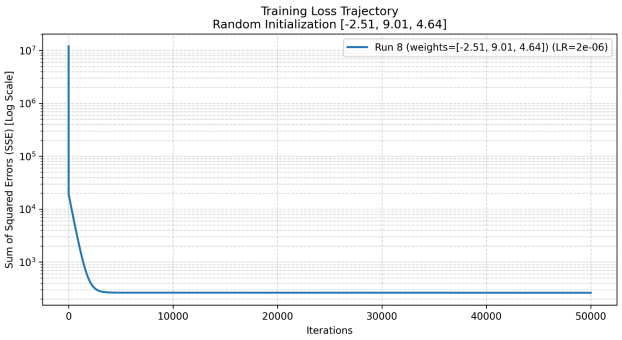
Iteration (0-indexed)	Sum of Squared Errors (SSE)
49999	262.1896

Stopping Condition: Max Iterations Reached (50,000 iterations). The model was approaching the minimum, but improving too slowly.

Final Weights: w[0] = -3.04, w[1] = -0.59, w[2] = 0.46

Acceptable or not: This model is somewhat acceptable. It does not have the lowest error but the error is pretty small and it was converging. Not optimal though.

Graph:



Run 9 (weights=[-2.51, 9.01, 4.64]) (LR=0.00000002)

Iteration (0-indexed)	Sum of Squared Errors (SSE)
0	11981078.1953
1	11724889.1458
5	10753838.9375
10	9652509.9411
50	4071312.1626
100	1391875.7447
500	19316.4943

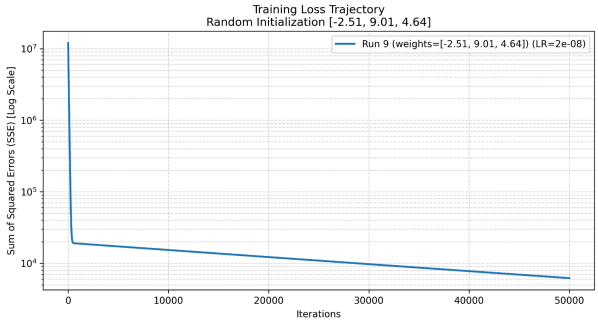
Iteration (0-indexed)	Sum of Squared Errors (SSE)
1000	18860.6558
5000	17206.2708
10000	15343.7006
25000	10897.0059
49999	6203.8004

Stopping Condition: Max Iterations Reached (50,000 iterations). The model was approaching the minimum, but improving too slowly.

Final Weights: w[0] = -2.58, w[1] = 3.74, w[2] = -0.79

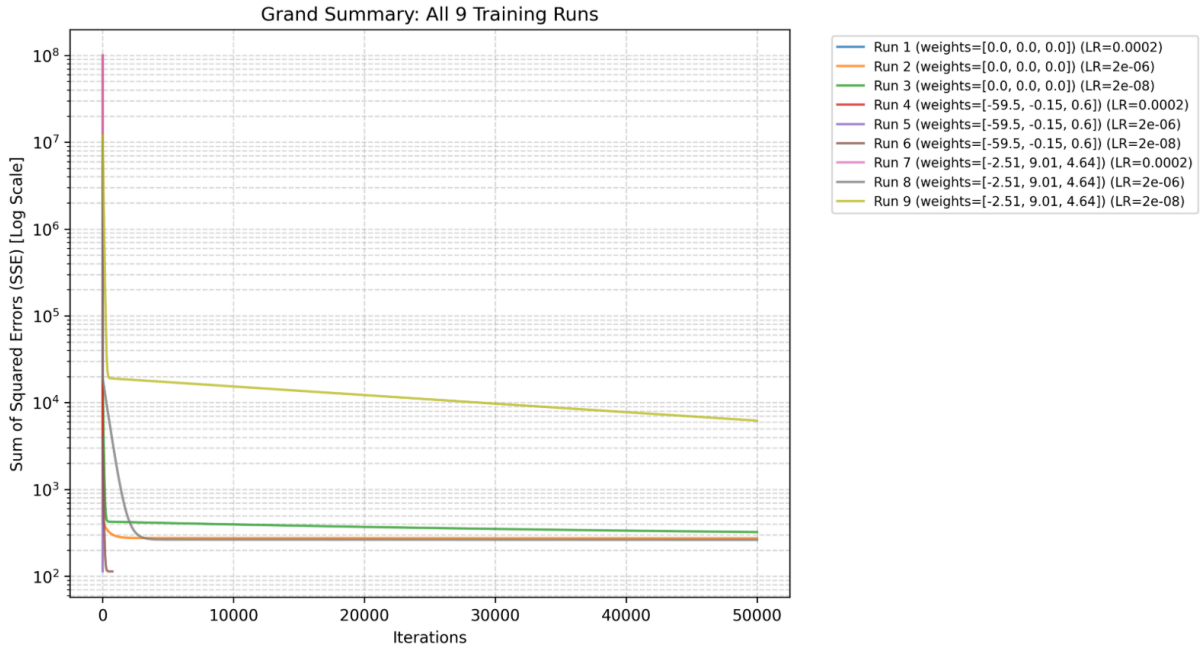
Acceptable or not: This model is not acceptable. It was converging pretty quickly so it may have found the minimum but the error is still pretty high.

Graph:



Training Summary and Best Result

I graphed all 9 runs' errors over the course of the 50000 iterations (to some degree of success) and here is the result.



The best model was **run 5** with starting weights of w[0] = -59.5, w[1] = -0.15, w[2] = 0.60 and a learning rate of 0.000002. It had the lowest SSE of 113.6609 and final weights of **w[0] = -59.5, w[1] = –0.12, w[2] = 0.72.**

Using this run, the regression model equation is:
$OXYCON = -59.5 - 0.12 * AGE + 0.72 * HEARTRATE$

I ran this model of all of the training dataset items and compared the predicted value (using the above equation) to the actual values. This is shown in the table below:

ID	AGE	HEARTRATE	OXYCON	Predicted OXYCON	Residual Error (Actual - Pred)	Squared Error
1	41	138	37.99	34.94	3.05	9.3025
2	42	153	47.34	45.62	1.72	2.9584
3	37	151	44.38	44.78	-0.40	0.1600
4	34	133	28.17	32.18	-4.01	16.0801
5	48	126	27.07	25.46	1.61	2.5921
6	44	145	37.85	39.62	-1.77	3.1329
7	43	158	44.72	49.10	-4.38	19.1844
8	46	143	36.42	37.94	-1.52	2.3104
9	37	138	31.21	35.42	-4.21	17.7241
10	38	158	54.85	49.70	5.15	26.5225

11	43	143	39.84	38.30	1.54	2.3716
12	43	138	30.83	34.70	-3.87	14.9769

As you can see, the model was off by a few units each time, more often predicting the OXYCON to be higher than it actually was.

While I am not an expert in the domain, my assumption is that this is at least better than predicting OXYCON to be lower than it was because now the estimate accounts for extra oxygen for the astronauts, so they likely have more than enough.

Discussion

An important thing to note is that our error surface is the sum of squared errors over a three variable regression model. This error surface has one minimum (no local minimums other than the absolute one) and is a paraboloid (or ‘bowl’) shape.

From the nine training runs, we can see some patterns emerge:

- One pattern is that the runs with initial weights of w[0] = -59.5, w[1] = -0.15, and w[2] = 0.60 (runs 4-6) were much more likely to reach the local minimum, probably because the initial weights were not far from the minimum and the initial gradient was not that high (as the further from the minimum, the steeper the slope, given our ‘bowl’ shape).

- Another pattern is that models with a higher learning rate of 0.0002 (runs 1, 4, and 7) diverged every time. This is because the model would calculate the initial gradient (which was likely pretty steep given our variables and the ‘bowl’ shape) and would multiply it with the learning rate before applying it to the weights. This caused the weights to update drastically, as the learning rate was not small enough to mitigate the effects of a large gradient, causing the point to move even further from the minimum and for the gradient to get even bigger, causing the error to diverge to infinity.

- The last pattern I noticed is that with smaller learning rates such as 0.000002 and 0.00000002, if the initial weights were far from the minimum (runs 2, 3, 8, and 9), the error would start approaching the local minimum but would take too long to get there, meaning the error was likely oscillating back and forth around the minimum, making its way down slowly. This is again because the initial weights being far from the minimum means the gradient is very large, and while the learning rates were high enough to stop the model from diverging, it still takes a very long time to dampen that large gradient and get down to the local minimum.

What I learned from this is that the placement of the initial weights is almost as important as the learning rate when it comes to training regression models where the error surface has one minimum.